

Tor Nelson

Seattle, Washington • (425) 686-8842 • tor.nelson@gmail.com

linkedin.com/in/tornelson • onmission.systems

Automation and systems engineer. Seven years building production automation and monitoring for a managed-services practice, implementing failure detection, self-healing remediation, and automated operations replacing manual administration across every client environment. For the last two years, the same reliability engineering discipline has guided the development of AI agent harnesses and the design of Human-AI collaboration systems, including a software development harness. The specification-first, review-gated system, with generated tests enforcing the behavioral contract, produced a production SaaS, now in final pre-launch. The through-line is: instrument the system, identify the failure modes, and build the machinery that increases reliability.

Working stack: PowerShell • Python • TypeScript / JavaScript • React • Node/Express • PostgreSQL • Windows Server • Microsoft 365 / Exchange Online • RMM (N-able N-central) • Vitest • Git

Experience

Founder / Systems & Product Engineer — OnMission Systems

Seattle, WA • 2025 – Present (*concurrent with Prairie IT Services*)

Designed and built **Copybook**, a production SaaS for designing intentional AI interactions and developing AI literacy through deliberate practice. Also developed the AI software development harness that produced the Copybook application. Work is demonstrable on request.

- **Designed and operated an AI software development harness.** The multi-stage system corrects for AI default behaviors and failure modes to improve reliability. Artifacts pass through review gates at each phase boundary, with generated unit, integration, and end-to-end tests acting as the behavioral contract, and the work stays auditable through decision records and validation reports. Review is calibrated rather than fixed, with demonstrated reliability driving increasing automation over time: full line-by-line review of generated code in the early phases, relaxing as the system proved itself.

- **Built the product itself to production standard** — TypeScript end to end: React 18 + Vite, Express, Drizzle ORM on Neon serverless Postgres, Zod, Zustand, TanStack Query, Stripe payments, Passport auth (local + Google OAuth), helmet / CSRF / rate-limiting middleware, Winston logging. Carries 1,478 automated test cases across 165 test files (Vitest, Testing Library, Supertest).
- **Designed a methodology for building AI-collaboration workflows** with the software development workflow as the first fully developed example. Work in progress applies the same principles and design language to the design of workflows in two other domains of knowledge work. Lessons learned in one instantiation are portable to other workflows.

IT Consultant & Automation Engineer — Prairie IT Services (independent contractor)

Seattle, WA • May 2019 – Present

Two-person managed-services practice supporting roughly ten small-business clients and several hundred endpoints. Carried daily operations and end-user support, and owned the monitoring, automation, and scripting lane outright — self-initiated, from scripts through monitoring to closed-loop remediation. Every system below is still in production.

- **Built the detection layer.** Monitoring that opens tickets on early-warning conditions with the relevant diagnostic logs already attached, so work begins with the evidence in hand rather than a user's description of the symptom. Shifted the queue from user-reported breakage toward detected conditions.
- **Built closed-loop remediation.** Automated detection and repair of system file corruption — a recurring incident class resolved without a technician touching the machine.
- **Built backup reliability automation** — failure detection, restore validation, and automated recovery workflows, replacing after-the-fact discovery of failed backups.
- **Replaced manual administration with version-controlled operations.** A composable PowerShell function library for Microsoft 365 and Exchange Online administration, standardized across every client tenant — repeatable and reviewable instead of point-and-click.
- **Integrated monitoring and scheduled work with the ticketing system,** generating work orders automatically rather than by hand.
- **Established a recurring security review process** across the client base, including the procedure, its execution, and the supporting documentation.
- **Ran the operational surface underneath all of it** — Windows desktop and server administration, endpoint support and triage, RMM administration, patching, identity and

license lifecycle, and the ticket queue. No tier to escalate to: diagnosis and resolution are one person's job.

Technical

Automation & scripting — PowerShell (primary), Python, Bash; composable function library design, version-controlled operational tooling, scheduled and event-triggered automation

Reliability & monitoring — alerting and detection design, diagnostic capture, closed-loop remediation, backup verification and recovery, failure-mode analysis

AI systems engineering — development harness design, specification-driven implementation, review-gate architecture, generated test suites as behavioral contracts, prompt and instruction-set design, agent workflow design

Software engineering — TypeScript, React, Node/Express, PostgreSQL, REST APIs, automated testing (Vitest, Testing Library, Supertest), Git, spec-first and test-driven development

Infrastructure & operations — Windows desktop and server administration, Microsoft 365 and Exchange Online, identity and license lifecycle, endpoint management and remediation, RMM (N-able N-central), ticketing workflow design, runbook and procedure documentation, security review

Education

B.S., The Evergreen State College — Molecular Biology and Biochemistry

Continuing self-directed coursework in computer science, data science, mathematics, and design, 2012–present.